Instrumente și tehnici de bază în informatică
Curs 6

Controlul versiunilor

Problemă:

- Lucrăm la un proiect
- Începem să implementăm un feature nou
- Ne dăm seama că ideea nu funcționează → vrem să revenim la o versiune precedentă și să încercăm altă implementare

Problemă:

- Lucrăm în colaborare cu mai multe persoane care fac independent modificări asupra proiectului

Soluție: Sisteme de control al versiunilor

Git

- Cel mai întâlnit sistem de control al versiunilor
- Creat de Linus Torvalds în 2005
- Alte sisteme: cvs, mercurial, etc

Git

- Menține un istoric (sub forma de DAG) al versiunilor proiectului
- *Repository*: director de lucru care conține un director `.git`
- *Commit*: Nod în graf; stare a repository-ului salvată în istoric
  - Statutul fișierelor
  - Autor și data
  - Mesaj de commit
  - ID hash calculat din conținutul fișierelor ('content addressing')
  - Pointer către commit-ul părinte

Git - comenzi de bază

Înainte de prima folosire trebuie configurate:

- `git config -global user.name`
- `git config -global user.email "email@exemplu.com"`

Git - comenzi de bază

- `git init`: inițializează un repository în directorul curent (creează directorul `.git`)
- `git status`: arată starea repository-ului (modificări de la ultimul commit)
- `git add <file>`: spune `git` să urmărească modificările din `<file>` pentru un următor commit (staging)
- `git commit -m <message>`: creează un nou commit cu modificările din fișierele urmărite
- `git commit -a -m <message>`: `git add` pentru toate fișierele urmate de `git commit`
- `.gitignore`: listează fișiere care vor fi ignorate de git (fișiere obiect mari, etc)

Git - comenzi de bază

- `git diff`: arată modificările
  - `git diff`: director de lucru vs staged changes
  - `git diff -staged`: staged changes vs ultimul commit
  - `git diff HEAD`: director de lucru vs ultimul commit

  Vezi și comanda unix `diff(1)`
- `git log`: istoricul commit-urilor

Ramuri

- `git branch`: listează ramurile
- `git branch <branch-name>`: creează o ramură nouă
- `git checkout <branch-name>`: schimbă ramura curentă
- `git checkout -b <branch-name>`: `git branch` + `git checkout`

Merge

```
$ git checkout main
$ git merge greeting
```

Mai multe situații:

- Ramura `main` nu s-a mutat $\rightarrow$ nu se creează un commit nou
- Ambele ramuri s-au mutat independent $\rightarrow$ se creează un commit cu doi părinți conținând modificările cumulate
- Ambele ramuri modifică același fișier $\rightarrow$ merge conflict: git nu poate rezolva automat, trebuie să rezolvăm manual. Fișierele în cauză primesc indicatoare de conflict care indică diferențele intre versiuni.

Lucrul remote

- Lucru colaborativ între mai mulți utilizatori
- Cel mai frecvent, dar nu obligatoriu, prin sisteme centralizate de stocare a repository-urilor (Github, Gitlab, etc.)
- `git remote add origin https://github.com/user/repo.git`
- `git push -u origin main`: încarcă commit-urile locale
- `git fetch`: descarcă commit-urile
- `git pull`: `fetch` + `merge` $\rightarrow$ pot apărea conflicte

Lucrul remote

Situația 1: Repository-ul există și vrem să inițializăm local

```
$ git clone https://github.com/user/repo.git
```

(echivalent cu)

```
$ mkdir repo
$ cd repo
$ git init
$ git remote add origin https://github.com/user/repo.git
$ git fetch origin
$ git checkout -b main origin/main
```

Lucrul remote

Situația 2: Inițializăm un repository local și îl sincronizăm cu remote

```
$ git init
$ git add .
$ git commit -m "Initial commit"

# Presupunand ca repo a fost creat pe github
$ git remote add origin https://github.com/user/repo.git
```

Lucrul remote

Exemplu:

```
$ git fetch origin
$ git merge origin/main      # (sau direct git pull)
facem modificari
$ git add .
$ git commit -m "Implemented feature ..."
$ git push -u origin main
```

Lucrul remote

Pull requests:

- Funcționalitate a platformei, nu a git
- Cerere către administratori (maintainers) să incorporeze un branch personal în proiect (de obicei în `main`)
- branch → push → pull request → code review → aprobare → merge